

熊猫API框架

Demo 项目代码文档

PandaAPI Framework - Demo Project Documentation

PHP 语法高亮 | VS Code 暗色主题风格

版本: v1.0.0

日期: 2024年

目录

1. 项目概述
2. index.php - 主入口文件 (片段 1~17)
3. Controllers.php - 控制器示例 (片段 18)
4. config.php - 配置文件 (片段 19)
5. init_database.php - 数据库初始化 (片段 20)

1. 项目概述

1.1 项目简介

本 Demo 项目展示了熊猫API框架 (PandaAPI) 的所有功能使用方法，包括 PandaDB 数据库操作、缓存系统、路由定义、中间件使用、数据验证等。

1.2 文件结构

```
demo/
+-- index.<?php      # 主入口文件，包含所有API路由
+-- Controllers.<?php # 控制器类示例
+-- config.<?php     # 配置文件
+-- init_database.<?php # 数据库初始化脚本
+-- api_test.<?php   # API测试
+-- README.md       # 项目说明
```

1.3 主要功能

用户管理：增删改查、分页、缓存

文章管理：JOIN查询、浏览量统计

认证系统：登录、注册、Token

缓存操作：文件缓存、Redis缓存

数据库操作：ThinkPHP风格、PandaDB风格

事务处理：回调式、手动式

2. index.php

Demo 项目主入口文件，包含完整的 API 路由定义和所有功能示例

片段 1：文件头部和命名空间导入

```
<?php
/**
 * 熊猫API框架 (PandaAPI) 完整演示项目
 * 本Demo展示了框架的所有功能使用方法
 * 包含：PandaDB数据库操作、缓存系统、路由、中间件、验证器等
 */

require_once __DIR__ . '/../vendor/autoload.php';

use PandaAPI\Core\App;
use PandaAPI\Core\Request;
use PandaAPI\Core\Response;
use PandaAPI\Database\PandaDB;
use PandaAPI\Route\Route;
use PandaAPI\Cache\Cache;
use PandaAPI\Validation\Validator;
```

片段 2：数据库和缓存配置

```
// 配置数据库连接（使用 PandaDB）
PandaDB::config([
    'driver' => 'mysql',
    'host'    => '127.0.0.1',
    'port'    => 3306,
    'database' => 'pandaapi_demo',
    'username' => 'root',
    'password' => '',
    'charset' => 'utf8mb4',
    'prefix' => 'panda_',
]);

// 配置缓存（支持：file/redis/memcache/openresty）
Cache::config([
    'driver' => 'file',
    'file' => [
        'path' => __DIR__ . '/../runtime/cache',
    ],
    'default_ttl' => 3600,
]);

// 设置慢查询阈值（毫秒）
PandaDB::setSlowThreshold(100);
```

片段 3：数据库初始化函数

```
/**
 * 初始化数据库表结构
 * 实际项目中通常通过迁移工具管理
```

```

*/
function initDatabase()
{
    // 创建用户表
    $createUsersTable = "
        CREATE TABLE IF NOT EXISTS panda_users (
            id INT AUTO_INCREMENT PRIMARY KEY,
            name VARCHAR(50) NOT NULL,
            email VARCHAR(100) NOT NULL UNIQUE,
            password VARCHAR(255) NOT NULL,
            status TINYINT DEFAULT 1,
            created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
            updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
        ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
    ";

    // 创建文章表
    $createPostsTable = "
        CREATE TABLE IF NOT EXISTS panda_posts (
            id INT AUTO_INCREMENT PRIMARY KEY,
            user_id INT NOT NULL,
            title VARCHAR(200) NOT NULL,
            content TEXT,
            views INT DEFAULT 0,
            status TINYINT DEFAULT 1,
            created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
            FOREIGN KEY (user_id) REFERENCES panda_users(id)
        ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
    ";

    try {
        PandaDB::execute($createUsersTable);
        PandaDB::execute($createPostsTable);
        return true;
    } catch (Exception $e) {
        return false;
    }
}

```

片段 4：健康检查接口

```

/**
 * 健康检查接口
 * GET /api/v1/health
 */
Route::get('/api/v1/health', function() {
    return Response::success([
        'framework' => 'PandaAPI',
        'name'       => 'PandaAPI',
        'status'     => 'healthy',
        'time'       => date('Y-m-d H:i:s'),
        'version'    => '1.0.0',
        'db_queries' => PandaDB::getQueryCount(),
        'uptime'     => microtime(true) - $_SERVER['REQUEST_TIME_FLOAT'],
    ]);
});

```

片段 5：路由组定义

```
/**
 * API版本组 v1
 * 所有接口统一前缀 /api/v1，并启用 CORS 中间件
 */
Route::group(['prefix' => '/api/v1', 'middleware' => ['Cors']], function() {

    // 用户相关接口 (公开)
    // 文章相关接口 (公开)
    // 聚合查询示例
    // 缓存操作示例
    // 原生SQL查询示例
    // 事务示例
    // PandaDB 风格快捷方法示例

});
```

片段 6：用户列表接口（ThinkPHP 风格分页）

```
/**
 * 获取用户列表
 * GET /api/v1/users?page=1&per_page=15&status=1
 */
Route::get('/users', function() {
    $request = new Request();
    $page = (int)$request->query('page', 1);
    $perPage = (int)$request->query('per_page', 15);
    $status = $request->query('status');

    // 使用 PandaDB 构建查询
    $builder = PandaDB::table('users');

    // 条件筛选
    if ($status !== null) {
        $builder->where('status', (int)$status);
    }

    // 排序 + 分页查询
    $builder->order('created_at', 'desc');
    $result = $builder->paginate($perPage, $page);

    return Response::paginate(
        $result['data'],
        $result['total'],
        $result['current_page'],
        $result['per_page']
    );
});
```

片段 7：用户详情接口（缓存穿透回源）

```
/**
 * 获取单个用户
 * GET /api/v1/users/{id}
 *
 * 使用缓存示例：先查缓存，不存在则查数据库
```

```

*/
Route::get('/users/{id}', function($id) {
    $cacheKey = "user:{$id}";

    // Cache::remember 自动处理缓存穿透
    $user = Cache::remember($cacheKey, function() use ($id) {
        return PandaDB::table('users')
            ->where('id', (int)$id)
            ->find();
    }, 300); // 缓存5分钟

    if (!$user) {
        return Response::error('用户不存在', 404);
    }

    return Response::success($user);
})->whereNumber('id');

```

片段 8：创建用户接口（数据验证 + 唯一性检查）

```

/**
 * 创建用户
 * POST /api/v1/users
 */
Route::post('/users', function() {
    $request = new Request();
    $data = $request->all();

    // 数据验证
    $validator = new Validator($data, [
        'name' => 'required|string|min:2|max:50',
        'email' => 'required|email',
        'password' => 'required|string|min:6',
    ]);

    if ($validator->fails()) {
        return Response::json([
            'error' => '验证失败',
            'messages' => $validator->errors()
        ], 422);
    }

    // 检查邮箱唯一性
    $exists = PandaDB::table('users')
        ->where('email', $data['email'])
        ->count();

    if ($exists > 0) {
        return Response::error('邮箱已存在', 422);
    }

    // 密码加密 + 插入
    $data['password'] = password_hash($data['password'], PASSWORD_DEFAULT);
    $data['created_at'] = date('Y-m-d H:i:s');
    $id = PandaDB::table('users')->insert($data);

    // 返回创建的用户（移除密码字段）
    $user = PandaDB::table('users')->where('id', $id)->find();
    unset($user['password']);

```

```
    return Response::json($user, 201);
});
```

片段 9：更新和删除用户接口

```
/**
 * 更新用户 PUT /api/v1/users/{id}
 */
Route::put('/users/{id}', function($id) {
    $request = new Request();
    $data = $request->all();

    $validator = new Validator($data, [
        'name' => 'string|min:2|max:50',
        'email' => 'email',
    ]);

    if ($validator->fails()) {
        return Response::json(['errors' => $validator->errors()], 422);
    }

    $user = PandaDB::table('users')->where('id', (int)$id)->find();
    if (!$user) {
        return Response::error('aaaaa', 404);
    }

    if (!empty($data)) {
        $data['updated_at'] = date('Y-m-d H:i:s');
        PandaDB::table('users')->where('id', (int)$id)->update($data);
        Cache::forget("user:{$id}"); // 清除缓存
    }

    $user = PandaDB::table('users')->where('id', (int)$id)->find();
    unset($user['password']);
    return Response::success($user);
})->whereNumber('id');

/**
 * 删除用户 DELETE /api/v1/users/{id}
 */
Route::delete('/users/{id}', function($id) {
    $user = PandaDB::table('users')->where('id', (int)$id)->find();
    if (!$user) {
        return Response::error('aaaaa', 404);
    }

    PandaDB::table('users')->where('id', (int)$id)->delete();
    Cache::forget("user:{$id}");

    return Response::success(['message' => 'aaaa']);
})->whereNumber('id');
```

片段 10：文章列表（JOIN 关联查询）

```
/**
 * 获取文章列表
```



```

* GET /api/v1/posts?page=1&per_page=10
*/
Route::get('/posts', function() {
    $request = new Request();
    $page = (int)$request->query('page', 1);
    $perPage = (int)$request->query('per_page', 10);

    // JOIN查询示例：关联用户表获取作者名
    $result = PandaDB::table('posts')
        ->field('posts.*', users.name as author_name')
        ->join('users', 'posts.user_id = users.id')
        ->where('posts.status', 1)
        ->order('posts.created_at', 'desc')
        ->paginate($perPage, $page);

    return Response::paginate(
        $result['data'],
        $result['total'],
        $result['current_page'],
        $result['per_page']
    );
});

```

片段 11：聚合统计接口

```

/**
 * 统计接口
 * GET /api/v1/stats
 */
Route::get('/stats', function() {
    $stats = [
        'users_count'      => PandaDB::table('users')->count(),
        'posts_count'      => PandaDB::table('posts')->count(),
        'posts_views_sum' => PandaDB::table('posts')->sum('views'),
        'posts_avg_views' => round(PandaDB::table('posts')->avg('views'), 2),
        'active_users'     => PandaDB::table('users')
                           ->where('status', 1)->count(),
    ];

    return Response::success($stats);
});

```

片段 12：缓存操作示例

```

/**
 * 缓存测试接口
 * GET /api/v1/cache/test
 */
Route::get('/cache/test', function() {
    $key = 'test:counter';

    // 原子递增
    if (!Cache::has($key)) {
        Cache::set($key, 0, 3600);
    }
    $count = Cache::increment($key);

```

```
// 批量操作
Cache::setMultiple([
    'cache:item1' => 'value1',
    'cache:item2' => 'value2',
    'cache:item3' => ['array' => 'data'],
], 600);

$items = Cache::getMultiple([
    'cache:item1', 'cache:item2', 'cache:item3'
]);

return Response::success([
    'counter' => $count,
    'items' => $items,
]);
});
```

片段 13 : 原生 SQL 查询

```
/**
 * 原生SQL查询
 * GET /api/v1/sql
 */
Route::get('/sql', function() {
    // 原生查询 (带参数绑定, 防止SQL注入)
    $users = PandaDB::query(
        "SELECT * FROM panda_users
        WHERE status = ?
        ORDER BY created_at DESC
        LIMIT 10",
        [1]
    );

    // 原生聚合查询
    $avgViews = PandaDB::query(
        "SELECT AVG(views) as avg_views
        FROM panda_posts
        WHERE status = 1"
    );

    return Response::success([
        'users' => $users,
        'avg_views' => $avgViews[0]['avg_views'] ?? 0,
    ]);
});
```

片段 14 : 事务操作示例

```
/**
 * 事务操作示例
 * POST /api/v1/transaction
 */
Route::post('/transaction', function() {
    try {
        // PandaDB::transaction 自动提交/回滚
        $result = PandaDB::transaction(function() {
            // 创建用户

```

```

        $userId = PandaDB::table('users')->insert([
            'name'      => 'aaaaaa',
            'email'     => 'transaction_' . time() . '@test.com',
            'password' => password_hash('123456', PASSWORD_DEFAULT),
            'status'    => 1,
            'created_at' => date('Y-m-d H:i:s'),
        ]);

// 创建用户的默认文章
PandaDB::table('posts')->insert([
    'user_id'    => $userId,
    'title'      => 'aaaa',
    'content'    => 'aaaaaaaaaa',
    'status'     => 1,
    'created_at' => date('Y-m-d H:i:s'),
]);

return ['user_id' => $userId];
});

return Response::success($result);
} catch (Exception $e) {
    return Response::error('aaaa: ' . $e->getMessage(), 500);
}
});

```

片段 15 : PandaDB 风格快捷方法

```

/**
 * PandaDB 双风格操作演示
 * GET /api/v1/pandadb-demo
 */
Route::get('/pandadb-demo', function() {
    // PandaDB 风格快捷方法
    $user = PandaDB::get(
        'users',
        ['id', 'name', 'email'],
        ['id' => 1]
    );

    // ThinkPHP 风格链式调用
    $users = PandaDB::table('users')
        ->where('status', 1)
        ->order('id', 'desc')
        ->limit(5)
        ->select();

    // name() 自动添加表前缀
    $count = PandaDB::name('users')->count();

    return Response::success([
        'single_user' => $user,
        'active_users' => $users,
        'total_count' => $count,
    ]);
});

```

片段 16 : 用户登录接口

```
/**
 * 用户登录
 * POST /api/v1/auth/login
 */
Route::post('/login', function() {
    $request = new Request();
    $data = $request->all();

    if (empty($data['email']) || empty($data['password'])) {
        return Response::error('00000000', 400);
    }

    // 查找用户
    $user = PandaDB::table('users')
        ->where('email', $data['email'])
        ->find();

    if (!$user) {
        return Response::error('00000000', 401);
    }

    // 验证密码
    if (!password_verify($data['password'], $user['password'])) {
        return Response::error('00000000', 401);
    }

    // 生成Token并存入缓存 (7天有效期)
    $token = bin2hex(random_bytes(32));
    Cache::set("token:{$token}", [
        'user_id' => $user['id'],
        'email' => $user['email'],
        'created_at' => time(),
    ], 86400 * 7);

    return Response::success([
        'token' => $token,
        'user' => [
            'id' => $user['id'],
            'name' => $user['name'],
            'email' => $user['email'],
        ],
    ]);
});
```

片段 17 : 应用启动

```
// 创建应用实例
$app = new App();

// 配置应用
$app->config([
    'env' => 'development',
    'debug' => true,
    'timezone' => 'Asia/Shanghai',
    'middleware' => ['Cors', 'Log'],
]);
```

```
// 运行  
$app->run();
```

3. Controllers.php

控制器类示例，展示如何使用控制器组织代码

片段 18 : UserController 完整控制器

```
<?php
namespace Demo\Controller;

use PandaAPI\Core\Request;
use PandaAPI\Core\Response;
use PandaAPI\Database\PandaDB;
use PandaAPI\Cache\Cache;
use PandaAPI\Validation\Validator;

class UserController
{
    /**
     * 用户列表 GET /users
     */
    public function index(Request $request)
    {
        $page = (int)$request->query('page', 1);
        $perPage = (int)$request->query('per_page', 15);
        $status = $request->query('status');

        $builder = PandaDB::table('users');
        if ($status !== null) {
            $builder->where('status', (int)$status);
        }

        $result = $builder->paginate($perPage, $page);

        return Response::paginate(
            $result['data'],
            $result['total'],
            $result['current_page'],
            $result['per_page']
        );
    }

    /**
     * 用户详情 GET /users/{id}
     */
    public function show(Request $request, $id)
    {
        $cacheKey = "user:{$id}";

        $user = Cache::remember($cacheKey, function() use ($id) {
            return PandaDB::table('users')
                ->where('id', (int)$id)
                ->find();
        }, 300);

        if (!$user) {
            return Response::error('用户不存在', 404);
        }
    }
}
```

```

        unset($user['password']);
        return Response::success($user);
    }

/**
 * 创建用户 POST /users
 */
public function store(Request $request)
{
    $data = $request->all();

    $validator = new Validator($data, [
        'name'      => 'required|string|min:2|max:50',
        'email'     => 'required|email',
        'password' => 'required|string|min:6',
    ]);

    if ($validator->fails()) {
        return Response::json([
            'error'      => '失败',
            'messages' => $validator->errors()
        ], 422);
    }

    $exists = PandaDB::table('users')
        ->where('email', $data['email'])
        ->count();

    if ($exists > 0) {
        return Response::error('失败', 422);
    }

    $data['password'] = password_hash($data['password'], PASSWORD_DEFAULT);
    $data['status']    = 1;
    $data['created_at'] = date('Y-m-d H:i:s');

    $id = PandaDB::table('users')->insert($data);

    $user = PandaDB::table('users')->where('id', $id)->find();
    unset($user['password']);

    return Response::json($user, 201);
}

/**
 * 更新用户 PUT /users/{id}
 */
public function update(Request $request, $id)
{
    $data = $request->all();
    $user = PandaDB::table('users')->where('id', (int)$id)->find();

    if (!$user) {
        return Response::error('失败', 404);
    }

    if (!empty($data)) {
        $data['updated_at'] = date('Y-m-d H:i:s');
        PandaDB::table('users')->where('id', (int)$id)->update($data);
        Cache::forget("user:{$id}");
    }
}

```

```
        $user = PandaDB::table('users')->where('id', (int)$id)->find();
        unset($user['password']);
        return Response::success($user);
    }

/**
 * 删除用户 DELETE /users/{id}
 */
public function destroy(Request $request, $id)
{
    $user = PandaDB::table('users')->where('id', (int)$id)->find();

    if (!$user) {
        return Response::error('用户不存在', 404);
    }

    PandaDB::table('users')->where('id', (int)$id)->delete();
    Cache::forget("user:{$id}");

    return Response::success(['message' => '删除成功']);
}
}
```


4. config.php

配置文件示例，包含数据库、缓存、中间件等配置

片段 19：完整配置文件

```
<?php
return [
    // 环境配置
    'env'      => 'development',
    'debug'    => true,
    'timezone' => 'Asia/Shanghai',

    // 数据库配置
    'database' => [
        'driver' => 'mysql',
        'host'   => '127.0.0.1',
        'port'   => 3306,
        'database' => 'pandaapi_demo',
        'username' => 'root',
        'password' => '',
        'charset' => 'utf8mb4',
        'prefix' => 'panda_',
    ],

    // 缓存配置
    'cache' => [
        'driver' => 'file', // file / redis / memcache / openresty
        'file' => [
            'path' => __DIR__ . '/../runtime/cache',
        ],
        'redis' => [
            'host'      => '127.0.0.1',
            'port'      => 6379,
            'password' => null,
            'database' => 0,
            'prefix' => 'pandaapi:',
        ],
        'default_ttl' => 3600,
    ],

    // 全局中间件
    'middleware' => [
        'Cors', // 跨域
        'Log',   // 日志
    ],

    // 限流配置
    'throttle' => [
        'max_attempts' => 60,
        'decay_minutes' => 1,
    ],

    // CORS配置
    'cors' => [
        'origin' => '*',
        'methods' => 'GET, POST, PUT, DELETE, PATCH, OPTIONS',
        'headers' => 'Content-Type, Authorization, X-Requested-With',
    ],
];
```

```
],  
  
// 慢查询阈值(毫秒)  
  'slow_query_threshold' => 100,  
];
```

5. init_database.php

数据库初始化脚本，创建表结构和示例数据

片段 20：数据库初始化脚本

```
<?php
/**
 * 数据库初始化脚本
 * 运行: php init_database.php
 */

require_once __DIR__ . '/../vendor/autoload.php';

use PandaAPI\Database\PandaDB;
use PandaAPI\Cache\Cache;

echo "PandaAPI Demo 初始化\n";

// 配置数据库
PandaDB::config([
    'driver' => 'mysql',
    'host'    => '127.0.0.1',
    'port'    => 3306,
    'database' => 'pandaapi_demo',
    'username' => 'root',
    'password' => '',
    'charset' => 'utf8mb4',
    'prefix' => 'panda_',
]);

try {
    // 创建用户表
    PandaDB::execute("
        CREATE TABLE IF NOT EXISTS panda_users (
            id INT AUTO_INCREMENT PRIMARY KEY,
            name VARCHAR(50) NOT NULL,
            email VARCHAR(100) NOT NULL UNIQUE,
            password VARCHAR(255) NOT NULL,
            status TINYINT DEFAULT 1,
            created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
            updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
            INDEX idx_status (status),
            INDEX idx_email (email)
        ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='用户表'
    ");
    echo "OK: 初始化用户表\n";

    // 创建文章表
    PandaDB::execute("
        CREATE TABLE IF NOT EXISTS panda_posts (
            id INT AUTO_INCREMENT PRIMARY KEY,
            user_id INT NOT NULL,
            title VARCHAR(200) NOT NULL,
            content TEXT,
            views INT DEFAULT 0,
            status TINYINT DEFAULT 1,
            created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
        ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='文章表'
    ");
    echo "OK: 初始化文章表\n";
}
```

```

        INDEX idx_user_id (user_id),
        INDEX idx_status (status),
        FOREIGN KEY (user_id) REFERENCES panda_users(id) ON DELETE CASCADE
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='用户'
);
echo "OK: 用户表创建成功\n";

// 插入示例用户
$users = [
    ['name' => '张三', 'email' => 'zhangsan@example.com',
      'password' => password_hash('123456', PASSWORD_DEFAULT)],
    ['name' => 'lisi', 'email' => 'lisi@example.com',
      'password' => password_hash('123456', PASSWORD_DEFAULT)],
    ['name' => 'wangwu', 'email' => 'wangwu@example.com',
      'password' => password_hash('123456', PASSWORD_DEFAULT)],
];

$userIds = [];
foreach ($users as $user) {
    $user['created_at'] = date('Y-m-d H:i:s');
    $user['status'] = 1;
    $userIds[] = PandaDB::table('users')->insert($user);
}
echo "OK: 用户 " . count($userIds) . " 条数据插入成功\n";

// 插入示例文章
$articles = [
    ['title' => 'PHP 7.4 新特性',
      'content' => 'PHP 7.4 新特性介绍...', 'views' => 1250],
    ['title' => 'MySQL 性能优化',
      'content' => 'MySQL 性能优化技巧...', 'views' => 890],
    ['title' => 'Redis 缓存应用',
      'content' => 'Redis 缓存应用案例...', 'views' => 1560],
];

foreach ($articles as $index => $article) {
    $article['user_id'] = $userIds[array_rand($userIds)];
    $article['status'] = 1;
    $article['created_at'] = date('Y-m-d H:i:s', strtotime("-{$index} days"));
    PandaDB::table('posts')->insert($article);
}
echo "OK: 文章 " . count($articles) . " 条数据插入成功\n";

} catch (Exception $e) {
    echo "FAIL: " . $e->getMessage() . "\n";
    exit(1);
}

echo "\n数据库初始化完成\n";
echo "用户: zhangsan@example.com / 123456\n";

```

— 文档结束 —